



Flow-based Deep Generative Models (Part 2)

<https://hku-data8018.github.io/>

Bo Dai

Spring 2026



- Slides reference
 - A Visual Dive into Conditional Flow Matching, <https://dl.heeere.com/conditional-flow-matching/blog/conditional-flow-matching/>
 - Continuous Normalizing Flows, <https://voletiv.github.io/talks>
 - Flow Matching for Generative Modeling, <https://neurips.cc/virtual/2024/tutorial/99531>

Normalizing flow models

- **Normalizing**: change of variables gives a normalized density after applying an invertible transformation
- **Flow**: Invertible transformations can be composed with each other

$$\mathbf{z}_m = \mathbf{f}_\theta^m \circ \cdots \circ \mathbf{f}_\theta^1(\mathbf{z}_0) = \mathbf{f}_\theta^m(\mathbf{f}_\theta^{m-1}(\cdots(\mathbf{f}_\theta^1(\mathbf{z}_0)))) \triangleq \mathbf{f}_\theta(\mathbf{z}_0)$$

- Start with a simple distribution for $\mathbf{z}_0$ (e.g. Normal distribution)
- Apply a sequence of M invertible transformations to finally obtain $\mathbf{x} = \mathbf{z}_M$
- By change of variables

$$p_{\mathbf{x}}(\mathbf{x}; \theta) = p_{\mathbf{z}}(\mathbf{f}_\theta^{-1}(\mathbf{x})) \prod_{m=1}^M \left| \det \left(\frac{\partial (\mathbf{f}_\theta^m)^{-1}(\mathbf{z}_m)}{\partial \mathbf{z}_m} \right) \right|$$

Normalizing flow models

- Learning via **maximum likelihood** over the dataset $\mathcal{D}$

$$\max_{\theta} \log p_X(\mathcal{D}; \theta) = \sum_{\mathbf{x} \in \mathcal{D}} \log p_Z(\mathbf{f}_{\theta}^{-1}(\mathbf{x})) + \log \left| \det \left(\frac{\partial \mathbf{f}_{\theta}^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

- Exact likelihood evaluation** via inverse transformation $\mathbf{x} \rightarrow \mathbf{z}$ and change of variables formula
- Sampling** via forward transformation $\mathbf{z} \rightarrow \mathbf{x}$

$$\mathbf{z} \sim p_Z(\mathbf{z}) \quad \mathbf{x} = \mathbf{f}_{\theta}(\mathbf{z})$$

- Latent representations** inferred via inverse transformation

$$\mathbf{z} = \mathbf{f}_{\theta}^{-1}(\mathbf{x})$$

Key points for normalizing flows

- **Simple prior** $p_z(z)$ that allows for efficient sampling and tractable likelihood evaluation.
- **Invertible transformations with tractable evaluation:**
 - Likelihood evaluation requires efficient evaluation of $x \rightarrow z$ mapping
 - Sampling requires efficient evaluation of $z \rightarrow x$ mapping
- Computing likelihoods also require **the evaluation of determinants of $n \times n$ Jacobian matrices**, where n is the data dimensionality
 - Computing the determinant for an $n \times n$ matrix is **$O(n^3)$** , which is expensive
- **Transformations with special structured Jacobian matrix.** For example, the determinant of a triangular matrix is the product of the diagonal entries, i.e., an $O(n)$ operation

Architectural constraints on Normalizing Flows tends to hinder their **expressivity**

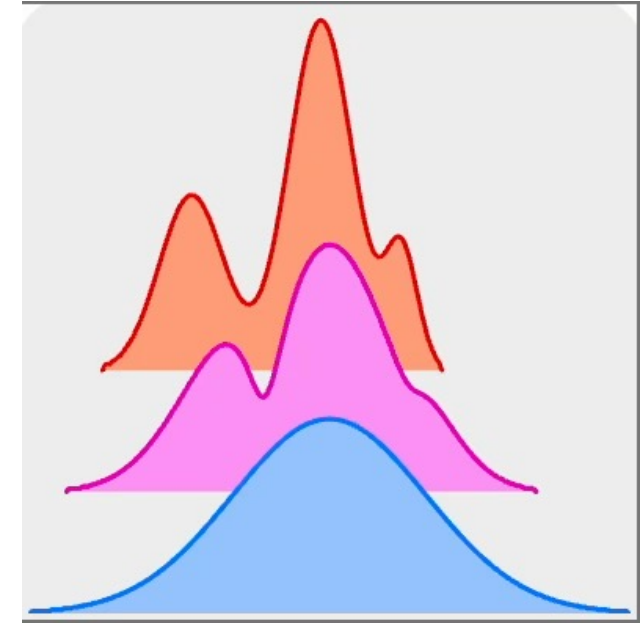
- **Continuous Normalizing Flows**
 - Discrete planar flow:

$$\phi_k(x) = x + \sigma(b_k^\top x + c)a_k$$

$$\begin{aligned} x_k &= \phi_k(x_{k-1}) \\ &= x_{k-1} + \sigma(b_k^\top x_{k-1} + c)a_k \\ &= x_{k-1} + \frac{1}{K}u_{k-1}(x_{k-1}) \end{aligned} \quad \hookrightarrow \quad u_{k-1}(\cdot) \stackrel{\text{def}}{=} K\sigma(b_k^\top \cdot + c)a_k$$

- Can be interpreted as a forward Euler discretization of the ODE

$$\begin{cases} x(0) = x_0 \\ \partial_t x(t) = u(x(t), t) \quad \forall t \in [0, 1] \end{cases}$$



Continuous Normalizing Flows

- ODE

$$\begin{cases} x(0) = x_0 \\ \partial_t x(t) = u(x(t), t) \quad \forall t \in [0, 1] \end{cases}$$

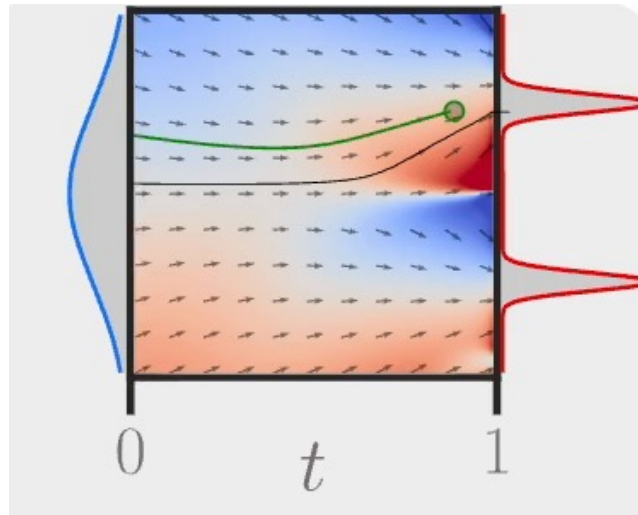
- **Inherently invertible** as one can solve the reverse-time ODE with $x(1) = T(x_0)$
- This ODE is called **an initial value problem**, controlled by **the velocity field u**

Continuous Normalizing Flows

- ODE

$$\begin{cases} x(0) = x_0 \\ \partial_t x(t) = u(x(t), t) \quad \forall t \in [0, 1] \end{cases}$$

- **Inherently invertible** as one can solve the reverse-time ODE with $x(1) = T(x_0)$
- This ODE is called **an initial value problem**, controlled by **the velocity field u**
- **Flow:** $f^u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, with $f^u(x, t)$ defined as the solution at time t to the initial value problem driven by u and initial condition $x(0) = x$

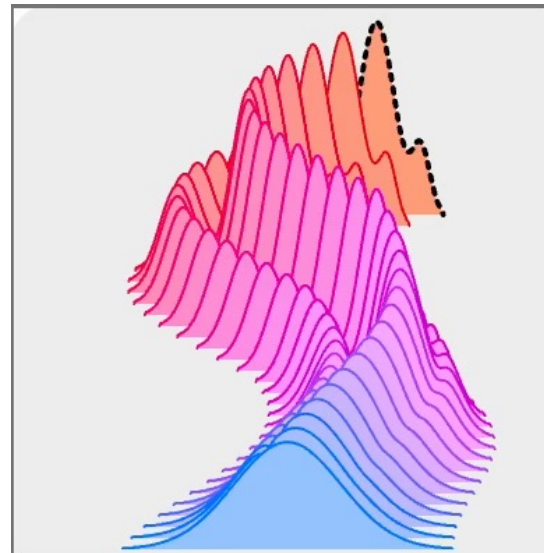


Continuous Normalizing Flows

- ODE

$$\begin{cases} x(0) = x_0 \\ \partial_t x(t) = u(x(t), t) \quad \forall t \in [0, 1] \end{cases}$$

- **Inherently invertible** as one can solve the reverse-time ODE with $x(1) = T(x_0)$
- This ODE is called **an initial value problem**, controlled by **the velocity field u**
- **Flow:** $f^u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, with $f^u(x, t)$ defined as the solution at time t to the initial value problem driven by u and initial condition $x(0) = x$
- **Probability (density) path:** $(p_t)_{t \in [0, 1]}$ is a time-dependent probability density function $\int p_t(x) dx = 1$
 - **An infinite number** of valid probability paths



Continuous Normalizing Flows

- ODE

$$\begin{cases} x(0) = x_0 \\ \partial_t x(t) = u(x(t), t) \quad \forall t \in [0, 1] \end{cases}$$

- Continuity equation:

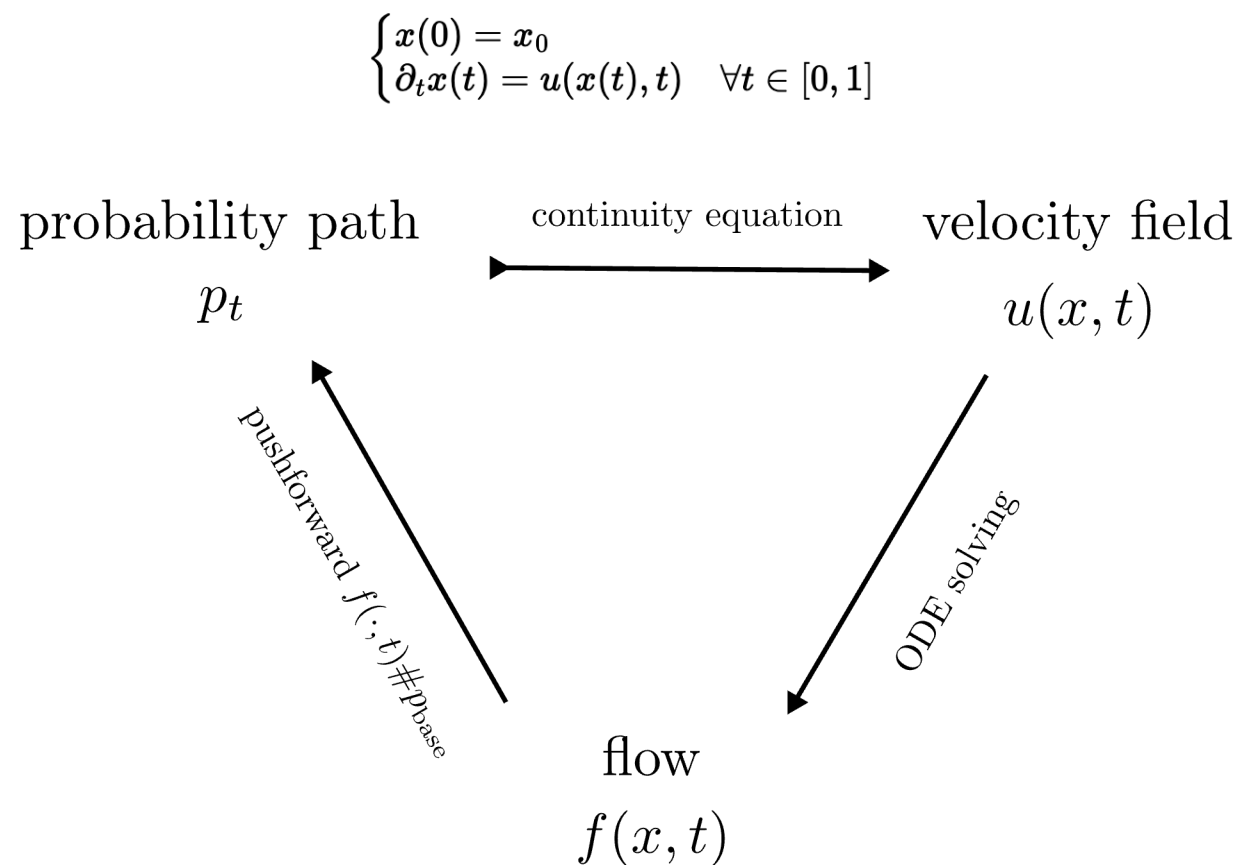
$$\partial_t p_t + \nabla \cdot u_t p_t = 0$$

- Probability mass is conserved, i.e., no new mass (dis)appears. Mathematically, it means that the change of the mass plus the change of the flux volume in all directions (a.k.a. the divergence of the flux) is constant.

$$\text{div}(V(x_1, \dots, x_D)) = \sum_{d=1}^D \frac{\partial V_d(x_1, \dots, x_D)}{\partial x_d}$$

Continuous Normalizing Flows

- ODE



Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

Solution:

$$x(t_1) = x(t_0) + \int_{t_0}^{t_1} f(x(t), t, \theta) dt$$

Example:

$$\frac{dx}{dt} = 2t; \quad x(0) = 2; \quad x(1) = ?$$

$$\begin{aligned} \Rightarrow x(1) &= x(0) + \int_0^1 2t dt \\ &= x(0) + (t^2|_{t=1} - t^2|_{t=0}) \\ &= 2 + 1^2 - 0^2 \\ &= 3 \end{aligned}$$

Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

Solution:

$$x(t_1) = x(t_0) + \int_{t_0}^{t_1} f(x(t), t, \theta) dt$$

What if this cannot be
analytically integrated?

Example:

$$\begin{aligned} \frac{dx}{dt} &= 2xt; \quad x(0) = 3 \\ \Rightarrow \int \frac{1}{2x} dx &= \int t dt \\ \Rightarrow \frac{1}{2} \log x &= \frac{1}{2} t^2 + c_0 \\ \Rightarrow x(t) &= ce^{t^2} \\ x(0) = 3 &\Rightarrow c = 2 \\ \therefore x(t) &= 2e^{t^2} \\ \Rightarrow x(1) &= 5.436 \end{aligned}$$

Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

Solution:

$$x(t_1) = x(t_0) + \int_{t_0}^{t_1} f(x(t), t, \theta) dt$$

Approximations to $\int_{t_0}^{t_1} f(x(t), t, \theta) dt$

i.e. Numerical Integration :

- Euler method
- Runge-Kutta methods
- ...

Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

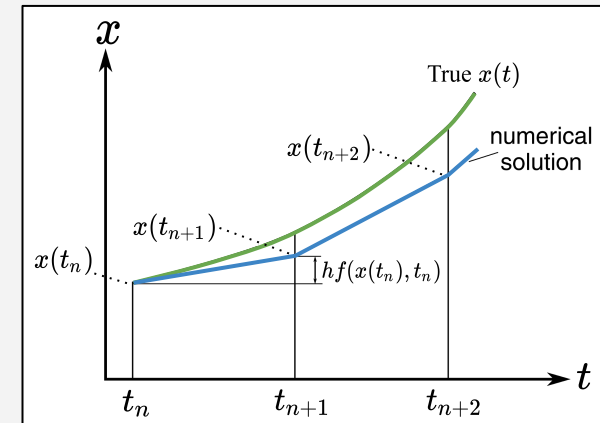
Solution:

$$x(t_1) = x(t_0) + \int_{t_0}^{t_1} f(x(t), t, \theta) dt$$

1st-order Runge-Kutta / Euler's method:

$$t_{n+1} = t_n + h \quad \text{-----} \rightarrow \text{Step size } h$$

$$x(t_{n+1}) = x(t_n) + hf(x(t_n), t_n) \quad \rightarrow \text{Update using derivative } f$$



<https://guide.freecodecamp.org/mathematics/differential-equations/eulers-method/>

Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

Solution:

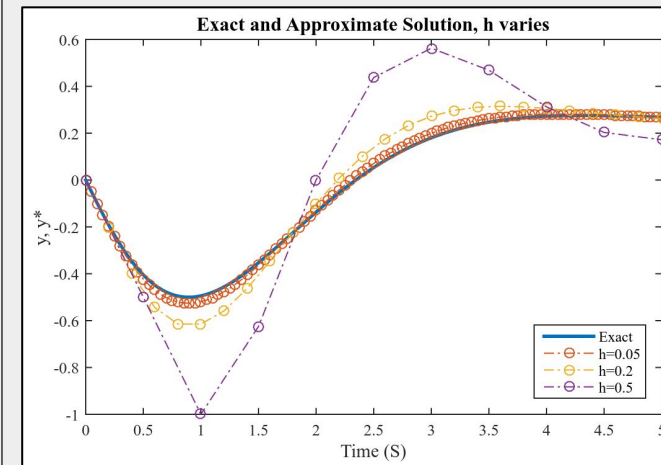
$$x(t_1) = x(t_0) + \int_{t_0}^{t_1} f(x(t), t, \theta) dt$$

1st-order Runge-Kutta / Euler's method:

$$t_{n+1} = t_n + h$$

$$x(t_{n+1}) = x(t_n) + hf(x(t_n), t_n)$$

Step size matters!



<https://lpsa.swarthmore.edu/NumInt/NumIntFirst.html>

Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

Solution:

$$x(t_1) = x(t_0) + \int_{t_0}^{t_1} f(x(t), t, \theta) dt$$



$$x(t_1) = \text{ODESolve}(f(x(t), t, \theta), x(t_0), t_0, t_1)$$

Final time

Initial time

Initial value

Differential

Any ODE solver of our choice!

Neural ODEs

Initial value problem:

$$\frac{dx(t)}{dt} = f(x(t), t, \theta); \quad x(t_0) \text{ is given}; \quad x(t_1) = ?$$

Solution:

$$x(t_1) = \text{ODESolve}(f(x(t), t, \theta), x(t_0), t_0, t_1)$$

f is a neural network!

Paradigm shift: whereas earlier f was pre-defined/hand-designed according to the domain, here we would like to estimate an f that suits our objective.

Neural ODEs

ODEs

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t), t, \theta)$$

Euler discretization

$$\mathbf{x}_{n+1} = \mathbf{x}_n + h f(\mathbf{x}_n, t_n, \theta)$$

Forward propagation:

$$\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)$$

$$L(\mathbf{x}(t_1)) \rightarrow \frac{\partial L}{\partial \theta} \quad \text{How to compute this?}$$

Update θ to reduce L

Residual networks

$$\mathbf{x}_{l+1} = \text{ResBlock}(\mathbf{x}_l, \theta)$$

$$\mathbf{x}_{l+1} = \mathbf{x}_l + g(\mathbf{x}_l, \theta)$$

Skip connection

$$\mathbf{y}_{pred} = \text{ResNet}(\mathbf{x})$$

Stacked ResBlocks

$$L(\mathbf{y}_{pred}) \rightarrow \frac{\partial L}{\partial \theta}$$

Update θ to reduce L

Neural ODEs

ODEs

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t), t, \theta)$$

Euler discretization

$$\mathbf{x}_{n+1} = \mathbf{x}_n + h f(\mathbf{x}_n, t_n, \theta)$$

Forward propagation:

$$\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)$$

$$L(\mathbf{x}(t_1)) \rightarrow \boxed{\frac{\partial L}{\partial \theta}}$$

Update θ to reduce L

Back-propagate through the ODE Solver!

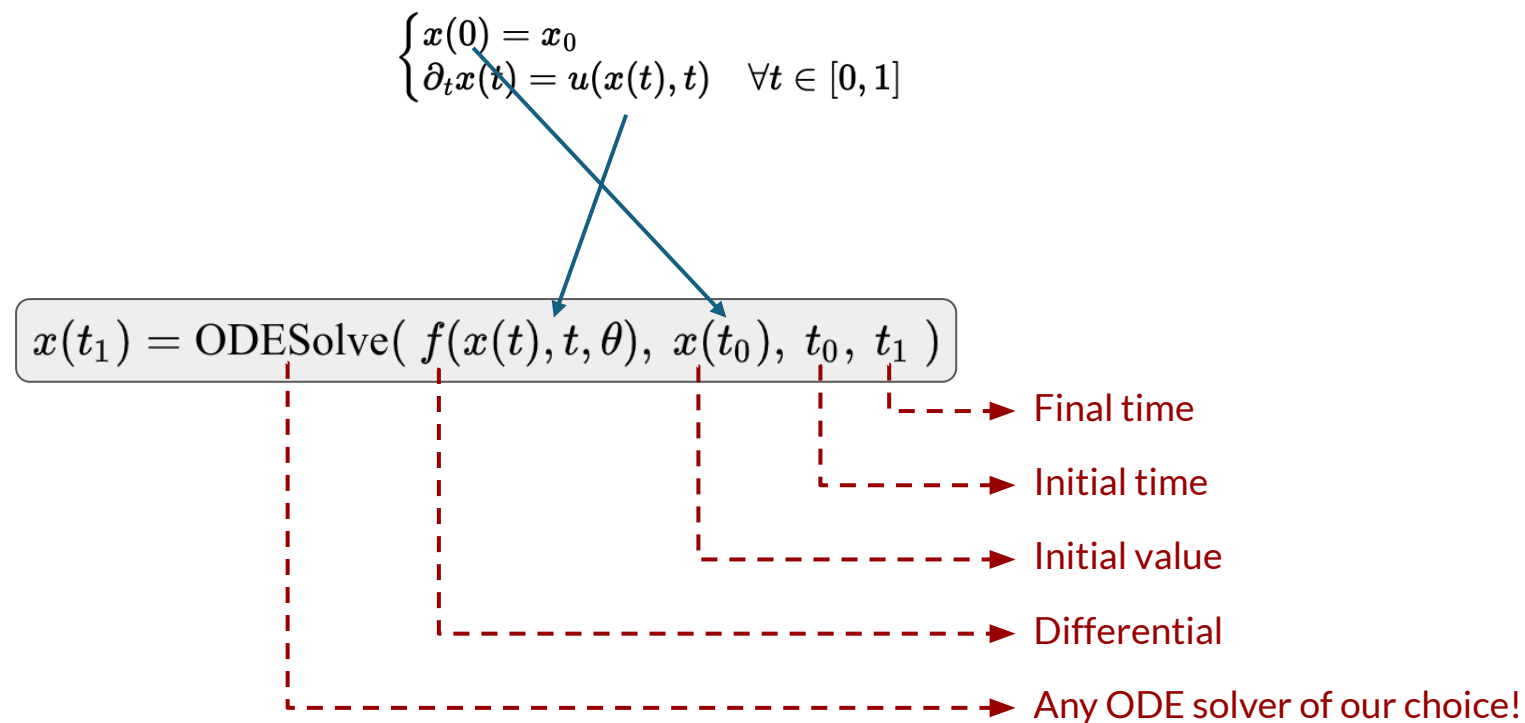
High memory cost -

need to save all activations of all iterations of ODESolve.

Can we do better?

Yes.

Continuous Normalizing Flows are **a particular case** of Neural ODE networks



Continuous Normalizing Flows are **a particular case** of Neural ODE networks

- Training is hard
 - **Expensive numerical ODE simulations** at training time
 - Estimators for the divergence **does not scale nicely** with high dimension
 - numerical approximations
 - the (infinite) number of possible probability paths
- **Benefits** over discrete cases
 - The constraints one needs to impose on u are **much less stringent** than in the discrete case
 - Inverting the flow can be achieved by simply **solving the ODE in reverse**
 - Computing the likelihood does not require inverting the flow, nor to compute a log determinant; **only the trace of the Jacobian is required.**

Flow matching: a **simulation-free** way to train continuous normalizing flows

- regression objective w.r.t. the parametric velocity field u_θ

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{x \sim p_t} [\|u_\theta(t, x) - u(t, x)\|^2].$$

$u(t, x)$ is the target velocity field inducing a probability path p_t

$$\log p_1(x) = \log p_0 - \int_0^1 (\nabla \cdot u_t)(x_t) dt.$$

Flow matching: a **simulation-free** way to train continuous normalizing flows

- regression objective w.r.t. the parametric velocity field u_θ

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{x \sim p_t} [\|u_\theta(t, x) - u(t, x)\|^2].$$

$u(t, x)$ is the target velocity field inducing a probability path p_t

- **Challenge:** requiring knowledge of a **valid $u(t, x)$** , which is our learning target.

Conditional flow matching:

- Express the probability path as **a marginal over a joint involving a latent variable z** :

$$p_t(x_t) = \int p(z) p_{t|z}(x_t | z) dz$$

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (u_t(x) p_t(x))$$

where $p_{t|z}(x_t | z)$ being a conditional probability path

Conditional flow matching:

- Express the probability path as **a marginal over a joint involving a latent variable z** :

$$p_t(x_t) = \int p(z) p_{t|z}(x_t | z) dz \qquad \frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (u_t(x) p_t(x))$$

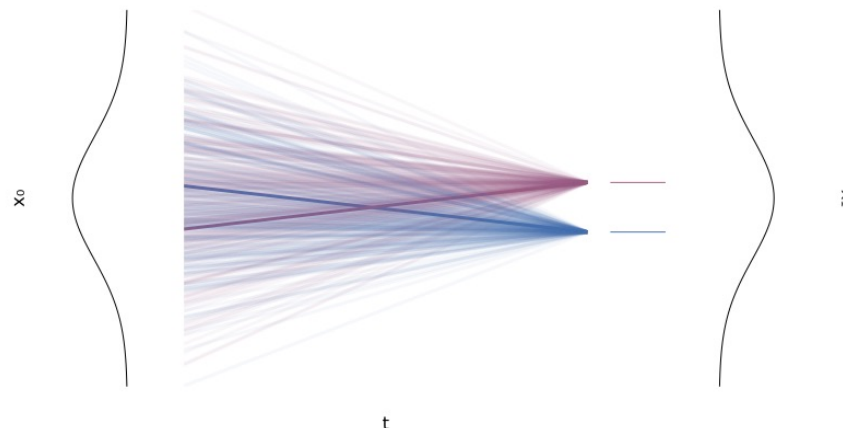
where $p_{t|z}(x_t | z)$ being a conditional probability path

- Since we have access to data samples. $x_1 \sim q_1$, it's reasonable to let $z = x_1$, leading to **the following marginal probability path**

$$p_t(x_t) = \int q_1(x_1) p_{t|1}(x_t | x_1) dx_1$$

the boundary conditions

$$p_0(x) = \mathcal{N}(x; 0, I) \quad p_0(x | x_1) = p_0 \quad \text{and} \quad p_1(x | x_1) = \mathcal{N}(x; x_1, \sigma_{\min}^2 I) \xrightarrow{\sigma_{\min} \rightarrow 0} \delta_{x_1}(x)$$



Conditional flow matching:

- $p_{t|z}(x_t | z)$ being a conditional probability path

The conditional probability path also satisfies the transport equation with the **conditional vector field** $u_t(x | x_1)$:

$$\frac{\partial p_t(x | x_1)}{\partial t} = -\nabla \cdot (u_t(x | x_1) p_t(x | x_1)). \quad (11)$$

Lipman et al. (2023) introduced the notion of **Conditional Flow Matching (CFM)** by noticing that this *conditional* vector field $u_t(x | x_1)$ can express the *marginal* vector $u_t(x)$ of interest via the conditional probability path $p_{t|1}(x_t | x_1)$ as

$$\begin{aligned} u_t(x) &= \mathbb{E}_{x_1 \sim p_{1|t}} [u_t(x | x_1)] \\ &= \int u_t(x | x_1) \frac{p_t(x | x_1) q_1(x_1)}{p_t(x)} dx_1. \end{aligned} \quad (12)$$

Conditional flow matching:

- $p_{t|z}(x_t | z)$ being a conditional probability path

$$\frac{\partial p_t(x | x_1)}{\partial t} = -\nabla \cdot (u_t(x | x_1) p_t(x | x_1)). \quad (11)$$

$$\begin{aligned} u_t(x) &= \mathbb{E}_{x_1 \sim p_1|t} [u_t(x | x_1)] \\ &= \int u_t(x | x_1) \frac{p_t(x | x_1) q_1(x_1)}{p_t(x)} dx_1. \end{aligned} \quad (12)$$

$$\begin{aligned} \frac{\partial p_t(x)}{\partial t} &= \frac{\partial}{\partial t} \int p_t(x | x_1) q(x_1) dx_1 \\ &= \int \frac{\partial}{\partial t} (p_t(x | x_1)) q(x_1) dx_1 \\ &= - \int \nabla \cdot (u_t(x | x_1) p_t(x | x_1)) q(x_1) dx_1 \\ &= - \int \nabla \cdot (u_t(x | x_1) p_t(x | x_1) q(x_1)) dx_1 \\ &= - \nabla \cdot \int u_t(x | x_1) p_t(x | x_1) q(x_1) dx_1 \\ &= - \nabla \cdot \left(\int u_t(x | x_1) \frac{p_t(x | x_1) q(x_1)}{p_t(x)} p_t(x) dx_1 \right) \\ &= - \nabla \cdot \left(\int u_t(x | x_1) \frac{p_t(x | x_1) q(x_1)}{p_t(x)} dx_1 p_t(x) \right) \\ &= - \nabla \cdot (u_t(x) p_t(x)) \end{aligned}$$

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (u_t(x) p_t(x)).$$

Conditional flow matching:

- $p_{t|z}(x_t | z)$ being a conditional probability path

$$\frac{\partial p_t(x | x_1)}{\partial t} = -\nabla \cdot (u_t(x | x_1)p_t(x | x_1)). \quad (11)$$

$$\begin{aligned} u_t(x) &= \mathbb{E}_{x_1 \sim p_{1|t}} [u_t(x | x_1)] \\ &= \int u_t(x | x_1) \frac{p_t(x | x_1)q_1(x_1)}{p_t(x)} dx_1. \end{aligned} \quad (12)$$

Moreover, equipped with the knowledge of (12), we can replace

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x \sim p_t} [\|u_\theta(t, x) - u(t, x)\|^2], \quad (13)$$

where $u_t(x) = \mathbb{E}_{x_1 \sim p_{1|t}} [u_t(x | x_1)]$, with an equivalent loss regressing the *conditional* vector field $u_t(x | x_1)$ and marginalising x_1 instead:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1 \sim q, x_t \sim p_t(x|x_1)} [\|u_\theta(t, x) - u_t(x | x_1)\|^2].$$

These losses are equivalent in the sense that

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta),$$

Conditional flow matching:

- $p_{t|z}(x_t | z)$ being a conditional probability path

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1 \sim q, x_t \sim p_t(x|x_1)} [\|u_\theta(t, x) - u_t(x | x_1)\|^2].$$

These losses are equivalent in the sense that

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta),$$

$$\|u_\theta(t, x) - u_t(x | x_1)\|^2 = \|u_\theta(t, x)\|^2 + \|u_t(x | x_1)\|^2 - 2\langle u_\theta(t, x), u_t(x | x_1) \rangle, \text{ and}$$

$$\|u_\theta(t, x) - u_t(x)\|^2 = \|u_\theta(t, x)\|^2 + \|u_t(x)\|^2 - 2\langle u_\theta(t, x), u_t(x) \rangle. \text{ Taking the expectation over the last inner}$$

$$\mathbb{E}_{x \sim p_t} \langle u_\theta(t, x), u_t(x) \rangle = \int \langle u_\theta(t, x), \int u_t(x|x_1) \frac{p_t(x | x_1)q(x_1)}{p_t(x)} dx_1 \rangle p_t(x) dx \quad (27)$$

$$= \int \langle u_\theta(t, x), \int u_t(x | x_1) p_t(x | x_1) q(x_1) dx_1 \rangle dx \quad (28)$$

$$= \int \int \langle u_\theta(t, x), u_t(x | x_1) \rangle p_t(x | x_1) q(x_1) dx_1 dx \quad (29)$$

$$= \mathbb{E}_{q_1(x_1)p(x|x_1)} \langle u_\theta(t, x), u_t(x | x_1) \rangle. \quad (30)$$

product term:

Then we see that the neural network squared norm terms are equal since:

$$\mathbb{E}_{p_t} \|u_\theta(t, x)\|^2 = \int \|u_\theta(t, x)\|^2 p_t(x | x_1) q(x_1) dx dx_1 = \mathbb{E}_{q_1(x_1)p(x|x_1)} \|u_\theta(t, x)\|^2 \quad \hookleftarrow$$

Conditional flow matching:

- $p_{t|z}(x_t | z)$ being a conditional probability path

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1 \sim q, x_t \sim p_t(x|x_1)} [\|u_\theta(t, x) - u_t(x | x_1)\|^2].$$

These losses are equivalent in the sense that

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta),$$

The benefit of the CFM loss is that once we define the conditional probability path $p_t(x | x_1)$, we can construct an unbiased Monte Carlo estimator of the objective using samples $(x_1^{(i)})_{i=1}^n$ from the data target q_1 !

This estimator can be efficiently computed as it involves an expectation over the joint $q_1(x_1)p_t(x | x_1)$, of the conditional vector field $u_t(x | x_1)$ both being available as opposed to the marginal vector field u_t which involves an expectation over the posterior $p_{1|t}(x_1 | x)$.

We note that, as opposed to the log-likelihood maximisation loss of CNFs which does not put any preference over which vector field u_t can be learned, the CFM loss does specify one via the choice of a conditional vector field, which will be regressed by the neural vector field u_θ .



To be continued